PANDUAN PRAKTIKUM & UJIAN SERKOM LARAVEL

Sistem Pemesanan Makanan (Restoran/Kafe) — Modul 1 s.d. 4 | Edisi Template Breeze Tailwind & Siap Ujian 21 Oktober

🎯 **TARGET: Lulus Sertifikasi Kompetensi Pemrograman Web BNSP / LSP (21 Oktober)**

🧭 TIMELINE & STRATEGI PENGERJAAN 4 JAM UJIAN

Alokasi Waktu	Modul & Target Utama	Capaian Fungsional Wajib
00:00 - 00:30 (30 Menit)	**Modul 1: Database & Model**	Konfigurasi .env, Migration 3 tabel (foods, orders, order_details), Model Relasi Eloquent, Seeder Menu & Akun Admin.
00:30 - 01:30 (60 Menit)	**Modul 2: Breeze Auth & CRUD Admin**	Install Breeze Blade, storage:link, FoodController (7 method CRUD + upload/hapus foto fisik), 3 View Admin berbalut template Tailwind Breeze.
01:30 - 02:30 (60 Menit)	**Modul 3: Sisi Customer & Transaksi**	OrderController::store (DB::transaction multi-tabel ACID), View Katalog Pelanggan modern Tailwind, Filter Kategori JS, Modal Konfirmasi Pesanan.
02:30 - 03:15 (45 Menit)	**Modul 4: Sisi Admin Rekap Pesanan**	Dashboard rekap transaksi (Eager Loading with()), Fitur Ubah Status Pesanan (PATCH), Badge status, Routing Lengkap routes/web.php.
03:15 - 04:00 (45 Menit)	**Finishing, Testing & Demo Asesor**	Uji fungsionalitas end-to-end, validasi form, upload foto, testing status, simulasi tanya jawab asesor (Grill-Me), cek kamus error jika ada kendala.

🎨 PANDUAN STYLING BREEZE & TAILWIND ESENSIAL UNTUK UJIAN

> 💡 **Manfaatkan Template Bawaan Laravel Breeze (Anti-Pusing!)**
>
> Setelah menjalankan `php artisan breeze:install blade` di Modul 2, proyek Anda sudah dilengkapi Tailwind CSS dan template bawaan. **Jangan mengetik styling dari nol!** Cukup contek class standar dari file bawaan Breeze:
>
> - **Card & Container Wrapper:** Contek dari `resources/views/dashboard.blade.php` → `py-12`, `max-w-7xl mx-auto sm:px-6 lg:px-8`, `bg-white overflow-hidden shadow-sm sm:rounded-lg p-6`.
> - **Form Input & Label:** Contek dari `resources/views/auth/login.blade.php` → `border-gray-300 focus:border-indigo-500 focus:ring-indigo-500 rounded-md shadow-sm w-full`.
> - **Tombol Aksi (Buttons):** Contek dari komponen Breeze → Tombol Utama: `px-4 py-2 bg-gray-800 text-white rounded-md text-sm font-semibold`, Tombol Hapus: `px-3 py-1.5 bg-red-600 text-white rounded-md text-xs font-semibold`, Tombol Batal: `px-4 py-2 bg-white border border-gray-300 text-gray-700 rounded-md text-sm font-semibold`.
> - **Tabel Rapi Breeze:** `min-w-full divide-y divide-gray-200`, thead `bg-gray-50 text-gray-500 text-xs font-medium uppercase`, row `hover:bg-gray-50`.

MODUL 1: SETUP DATABASE, MIGRATION, MODEL & SEEDER

Membangun fondasi database MySQL, struktur tabel dengan relasi foreign key, model Eloquent berelasi, dan data awal untuk pengujian.

📌 KAMUS WAJIB: KAPAN MEMAKAI Food, food, ATAU foods

> ⚠️ **Mengapa Masalah 'Food vs foods' Sering Memicu Error?**
>
> Kata '*food*' dalam bahasa Inggris adalah **uncountable noun** (kata benda tak terhitung). Secara default, generator Laravel terkadang menganggap jamak dari 'food' tetap 'food', sehingga saat Anda menjalankan `php artisan make:model Food -m`, Laravel bisa menghasilkan migrasi `create_food_table` (tunggal). Hal ini memicu error fatal foreign key pada `order_details` atau error `Table 'pesanmakan.foods' doesn't exist` saat Eloquent dijalankan.
>
> **4 Formula Emas Penamaan Laravel:**
>
> - **Nama Tabel MySQL:** Wajib jamak `foods` (berakhiran s) di migrasi: `Schema::create('foods', ...)`.
> - **Model Eloquent:** Wajib tegaskan `protected $table = 'foods';` di `app/Models/Food.php`.
> - **Kolom Foreign Key:** Tuliskan relasi secara eksplisit: `$table->foreignId('food_id')->constrained('foods')->cascadeOnDelete();` di tabel `order_details`.
> - **Variabel Controller:** Gunakan tunggal `$food` untuk 1 record menu dan jamak `$foods` untuk collection menu.

Objek	Penulisan	Contoh di Kode	Penjelasan
Model (PHP Class)	`Food (Kapital, Tunggal)`	`class Food extends Model` `Food::all()`	Representasi 1 entitas objek model Eloquent.
Tabel MySQL	`foods (Kecil, Jamak)`	`Schema::create('foods', ...)` `protected $table = 'foods';`	Nama fisik tabel database di MySQL (wajib berakhiran 's').
Kolom Foreign Key	`food_id (Kecil, Tunggal + _id)`	`$table->foreignId('food_id')->constrained('foods')`	Kolom relasi kunci asing di tabel order_details.
Variabel Tunggal	`$food (Kecil, Tunggal)`	`public function edit(Food $food)` `$food = Food::findOrFail($id);`	Menyimpan 1 objek record menu makanan.
Variabel Jamak	`$foods (Kecil, Jamak)`	`$foods = Food::latest()->paginate(10);`	Menyimpan kumpulan (collection) banyak menu makanan.

[1.1] .env

Buka file .env di root project. Sesuaikan nama database MySQL (buat database bernama 'pesanmakan' di phpMyAdmin terlebih dahulu).

Kondisi Awal Bawaan Generator Laravel — Default File

```
DB_CONNECTION=mysql
DB_HOST=127.0.0.1
DB_PORT=3306
DB_DATABASE=laravel
DB_USERNAME=root
DB_PASSWORD=
```

KODE YANG DITAMBAHKAN / DIKETIK PESERTA — Wajib Ditambahkan

```
DB_CONNECTION=mysql
DB_HOST=127.0.0.1
DB_PORT=3306
DB_DATABASE=pesanmakan    // <-- [UBAH] Ganti nama database menjadi pesanmakan
DB_USERNAME=root
DB_PASSWORD=              // <-- [SESUAIKAN] Kosongkan jika pakai XAMPP default
```

Bedah Baris & Kenapa Ditulis Begitu?

- DB_DATABASE=pesanmakan: Menghubungkan Laravel ke database proyek di MySQL phpMyAdmin.
- MySQL di control panel XAMPP/Laragon WAJIB dalam status RUNNING sebelum menjalankan migration apapun.

[1.2] Terminal Command (Generator Model, Migration, Controller)

Jalankan 3 perintah sakti di terminal untuk membuat Model, Migration, dan Controller sekaligus:

Kondisi Awal Bawaan Generator Laravel — Default File

```
# Belum ada file Model, Migration, dan Controller
```

KODE YANG DITAMBAHKAN / DIKETIK PESERTA — Wajib Ditambahkan

```
php artisan make:model Food -mcr
php artisan make:model Order -mcr
php artisan make:model OrderDetail -m
```

Bedah Baris & Kenapa Ditulis Begitu?

- Flag -m: Otomatis membuat file migration di database/migrations/ (Catatan: Karena 'food' kata benda tak terhitung/uncountable, cek file migrasi dan pastikan nama tabel fisik dijadikan 'foods').
- Flag -c: Otomatis membuat file controller di app/Http/Controllers/.
- Flag -r: Membuat controller bertipe Resource (otomatis menyediakan 7 method CRUD standar: index, create, store, show, edit, update, destroy).
- Model OrderDetail cukup flag -m saja karena fungsi transaksinya dipusatkan di dalam OrderController.

[1.3] database/migrations/xxxx_xx_xx_create_foods_table.php

Buka file migrasi tabel foods. Urutan eksekusi migration HARUS: foods -> orders -> order_details.

Kondisi Awal Bawaan Generator Laravel — Default File

```
public function up(): void
{
    Schema::create('foods', function (Blueprint $table) {
        $table->id();
        $table->timestamps();
    });
}
```

KODE YANG DITAMBAHKAN / DIKETIK PESERTA — Wajib Ditambahkan

```
public function up(): void
{
    Schema::create('foods', function (Blueprint $table) {
        $table->id();
        $table->string('name');                    // <-- [TAMBAHKAN] Nama menu
```

```
makanan/minuman
            $table->enum('category', ['Makanan', 'Minuman', 'Cemilan']); // <-- [TAMBAHKAN] Kategori terbatas
            $table->integer('price');                                    // <-- [TAMBAHKAN] Harga satuan
            $table->text('description');                                 // <-- [TAMBAHKAN] Deskripsi menu
            $table->string('image')->nullable();                         // <-- [TAMBAHKAN] Path foto (nullable jika
belum ada foto)
            $table->timestamps();
        });
    }
```

🔍 **Bedah Baris & Kenapa Ditulis Begitu?**

- KUNCI PALING KRUSIAL: Nama tabel HARUS jamak berakhiran s ('foods'). Di beberapa versi Laravel, generator model Food kadang menghasilkan 'food' (tunggal). Pastikan tertulis Schema::create('foods', ...) agar relasi foreignId('food_id')->constrained('foods') pada order_details tidak crash!
- image dibuat ->nullable(): Mengizinkan data makanan tersimpan walaupun peserta atau admin belum mengupload file foto.

[1.4] database/migrations/xxxx_xx_xx_create_orders_table.php

Buka file migrasi tabel orders (header transaksi pesanan meja).

📄 Kondisi Awal Bawaan Generator Laravel — Default File

```
public function up(): void
{
    Schema::create('orders', function (Blueprint $table) {
        $table->id();
        $table->timestamps();
    });
}
```

🖍️ KODE YANG DITAMBAHKAN / DIKETIK PESERTA — Wajib Ditambahkan

```
public function up(): void
{
    Schema::create('orders', function (Blueprint $table) {
        $table->id();
        $table->string('customer_name');                          // <-- [TAMBAHKAN] Nama pemesan
        $table->string('table_number');                           // <-- [TAMBAHKAN] Nomor meja makan
        $table->integer('total_price');                           // <-- [TAMBAHKAN] Akumulasi total harga
pesanan
        $table->enum('status', ['Pending', 'Diproses', 'Selesai'])
              ->default('Pending');                               // <-- [TAMBAHKAN] Status pesanan default
Pending
        $table->timestamps();
    });
}
```

🔍 **Bedah Baris & Kenapa Ditulis Begitu?**

- default('Pending'): Menjamin setiap transaksi baru dari customer langsung berstatus 'Pending' tanpa perlu diinput manual dari form pelanggan.

[1.5] database/migrations/xxxx_xx_xx_create_order_details_table.php

Buka file migrasi tabel order_details (rincian item menu yang dipesan).

Kondisi Awal Bawaan Generator Laravel — Default File

```php
public function up(): void
{
    Schema::create('order_details', function (Blueprint $table) {
        $table->id();
        $table->timestamps();
    });
}
```

KODE YANG DITAMBAHKAN / DIKETIK PESERTA — Wajib Ditambahkan

```php
public function up(): void
{
    Schema::create('order_details', function (Blueprint $table) {
        $table->id();
        $table->foreignId('order_id')
              ->constrained('orders')
              ->cascadeOnDelete();                          // <-- [TAMBAHKAN] Terhubung ke orders.id
        $table->foreignId('food_id')
              ->constrained('foods')
              ->cascadeOnDelete();                          // <-- [TAMBAHKAN] Terhubung ke foods.id
        $table->integer('quantity');                         // <-- [TAMBAHKAN] Jumlah porsi
        $table->integer('subtotal');                         // <-- [TAMBAHKAN] Subtotal = harga *
quantity
        $table->timestamps();
    });
}
```

Bedah Baris & Kenapa Ditulis Begitu?

- foreignId('order_id')->constrained('orders'): Membuat kolom foreign key yang mengunci integritas data ke tabel orders.
- foreignId('food_id')->constrained('foods'): Mengunci integritas ke tabel master 'foods'.
- cascadeOnDelete(): Jika data order dihapus, semua baris rincian di tabel order_details otomatis ikut terhapus bersih.

[1.6] app/Models/Food.php

Buka model Food. Tambahkan penegasan nama tabel fisik dan proteksi mass assignment guarded.

Kondisi Awal Bawaan Generator Laravel — Default File

```php
<?php

namespace App\Models;

use Illuminate\Database\Eloquent\Factories\HasFactory;
use Illuminate\Database\Eloquent\Model;

class Food extends Model
{
    use HasFactory;
}
```

KODE YANG DITAMBAHKAN / DIKETIK PESERTA — Wajib Ditambahkan

```php
<?php

namespace App\Models;

use Illuminate\Database\Eloquent\Factories\HasFactory;
```

```php
use Illuminate\Database\Eloquent\Model;

class Food extends Model
{
    use HasFactory;

    protected $table = 'foods';         // <-- [KUNCI] Wajib tegaskan nama tabel MySQL adalah 'foods'
    protected $guarded = ['id'];        // <-- [TAMBAHKAN] Izinkan semua kolom selain id diisi mass-
assignment
}
```

> 🔍 **Bedah Baris & Kenapa Ditulis Begitu?**
> - protected $table = 'foods': KUNCI PALING KRUSIAL! Kata 'food' adalah uncountable noun dalam bahasa Inggris, sehingga tanpa baris ini Eloquent mencari tabel 'food' (tunggal), memicu error: Table 'pesanmakan.foods' doesn't exist.
> - protected $guarded = ['id']: TRIK HAFALAN CEPAT SERKOM! Daripada mengetik 5 kolom di $fillable, cukup amankan 'id'. Seluruh field lain otomatis diizinkan untuk Food::create([...]).

[1.7] app/Models/Order.php

Buka model Order. Pasang proteksi guarded dan relasi One-to-Many ke OrderDetail.

📄 Kondisi Awal Bawaan Generator Laravel — Default File

```php
<?php

namespace App\Models;

use Illuminate\Database\Eloquent\Factories\HasFactory;
use Illuminate\Database\Eloquent\Model;

class Order extends Model
{
    use HasFactory;
}
```

🖍️ KODE YANG DITAMBAHKAN / DIKETIK PESERTA — Wajib Ditambahkan

```php
<?php

namespace App\Models;

use Illuminate\Database\Eloquent\Factories\HasFactory;
use Illuminate\Database\Eloquent\Model;

class Order extends Model
{
    use HasFactory;

    protected $guarded = ['id'];        // <-- [TAMBAHKAN]

    // Relasi: 1 Order memiliki BANYAK OrderDetail (One to Many)
    public function orderDetails()      // <-- [TAMBAHKAN]
    {
        return $this->hasMany(OrderDetail::class, 'order_id');
    }
}
```

> 🔍 **Bedah Baris & Kenapa Ditulis Begitu?**
>
> - hasMany(OrderDetail::class, 'order_id'): Memungkinkan controller memanggil $order->orderDetails untuk mengambil seluruh item pesanan milik order tersebut di tabel rekap admin.

[1.8] app/Models/OrderDetail.php

Buka model OrderDetail. Pasang proteksi guarded dan relasi balik belongsTo.

📄 Kondisi Awal Bawaan Generator Laravel — Default File

```php
<?php

namespace App\Models;

use Illuminate\Database\Eloquent\Factories\HasFactory;
use Illuminate\Database\Eloquent\Model;

class OrderDetail extends Model
{
    use HasFactory;
}
```

🖍️ KODE YANG DITAMBAHKAN / DIKETIK PESERTA — **Wajib Ditambahkan**

```php
<?php

namespace App\Models;

use Illuminate\Database\Eloquent\Factories\HasFactory;
use Illuminate\Database\Eloquent\Model;

class OrderDetail extends Model
{
    use HasFactory;

    protected $table = 'order_details'; // <-- [TAMBAHKAN]
    protected $guarded = ['id'];        // <-- [TAMBAHKAN]

    // Relasi balik ke Order induk
    public function order()             // <-- [TAMBAHKAN]
    {
        return $this->belongsTo(Order::class, 'order_id');
    }

    // Relasi balik ke Menu Makanan
    public function food()              // <-- [TAMBAHKAN]
    {
        return $this->belongsTo(Food::class, 'food_id');
    }
}
```

> 🔍 **Bedah Baris & Kenapa Ditulis Begitu?**
>
> - belongsTo(Food::class, 'food_id'): Sangat krusial! Memungkinkan dashboard admin memanggil $detail->food->name untuk mengetahui nama menu yang dipesan.

[1.9] database/seeders/FoodSeeder.php

Buat seeder makanan dengan terminal: php artisan make:seeder FoodSeeder. Lalu isi 5 data menu awal:

Kondisi Awal Bawaan Generator Laravel — Default File

```php
<?php

namespace Database\Seeders;

use Illuminate\Database\Console\Seeds\WithoutModelEvents;
use Illuminate\Database\Seeder;

class FoodSeeder extends Seeder
{
    /**
     * Run the database seeds.
     */
    public function run(): void
    {
        //
    }
}
```

KODE YANG DITAMBAHKAN / DIKETIK PESERTA — Wajib Ditambahkan

```php
<?php

namespace Database\Seeders;

use Illuminate\Database\Seeder;
use Illuminate\Support\Facades\DB; // <-- [TAMBAHKAN] Import Facade DB

class FoodSeeder extends Seeder
{
    public function run(): void
    {
        // <-- [TAMBAHKAN] Masukkan 5 data menu awal
        DB::table('foods')->insert([
            [
                'name'        => 'Nasi Goreng Spesial',
                'category'    => 'Makanan',
                'price'       => 25000,
                'description' => 'Nasi goreng telur + ayam suwir gurih.',
                'image'       => null,
                'created_at'  => now(),
                'updated_at'  => now(),
            ],
            [
                'name'        => 'Mie Goreng Seafood',
                'category'    => 'Makanan',
                'price'       => 28000,
                'description' => 'Mie goreng pedas cumi dan udang.',
                'image'       => null,
                'created_at'  => now(),
                'updated_at'  => now(),
            ],
            [
                'name'        => 'Es Teh Manis',
                'category'    => 'Minuman',
                'price'       => 5000,
                'description' => 'Es teh melati segar dingin.',
                'image'       => null,
                'created_at'  => now(),
                'updated_at'  => now(),
            ],
```

```
            [
                'name'        => 'Jus Alpukat',
                'category'    => 'Minuman',
                'price'       => 15000,
                'description' => 'Jus alpukat kental susu cokelat.',
                'image'       => null,
                'created_at'  => now(),
                'updated_at'  => now(),
            ],
            [
                'name'        => 'Kentang Goreng',
                'category'    => 'Cemilan',
                'price'       => 12000,
                'description' => 'Kentang renyah saus keju gurih.',
                'image'       => null,
                'created_at'  => now(),
                'updated_at'  => now(),
            ],
        ]);
    }
}
```

🔍 **Bedah Baris & Kenapa Ditulis Begitu?**

- now(): Mengisi timestamp created_at dan updated_at agar query Eloquent latest() tidak error.
- image diset null: Karena file fisik gambar belum diupload pada tahap awal seeding.

[1.10] database/seeders/DatabaseSeeder.php

Buka DatabaseSeeder.php untuk membuat Akun Login Admin dan memanggil FoodSeeder sekaligus.

📄 Kondisi Awal Bawaan Generator Laravel — Default File

```php
<?php

namespace Database\Seeders;

// use Illuminate\Database\Console\Seeds\WithoutModelEvents;
use Illuminate\Database\Seeder;

class DatabaseSeeder extends Seeder
{
    /**
     * Seed the application's database.
     */
    public function run(): void
    {
        // \App\Models\User::factory(10)->create();
    }
}
```

🖍️ KODE YANG DITAMBAHKAN / DIKETIK PESERTA — Wajib Ditambahkan

```php
<?php

namespace Database\Seeders;

use App\Models\User;                // <-- [TAMBAHKAN] Import Model User
use Illuminate\Database\Seeder;
use Illuminate\Support\Facades\Hash;  // <-- [TAMBAHKAN] Import Facade Hash
```

```php
class DatabaseSeeder extends Seeder
{
    public function run(): void
    {
        // 1. <-- [TAMBAHKAN] Akun Admin Default untuk Pengujian Login Asesor
        User::create([
            'name'     => 'Admin Toko',
            'email'    => 'admin@gmail.com',
            'password' => Hash::make('password123'),
        ]);

        // 2. <-- [TAMBAHKAN] Panggil Seeder Menu Makanan
        $this->call([
            FoodSeeder::class,
        ]);
    }
}
```

> 🔍 **Bedah Baris & Kenapa Ditulis Begitu?**
> - Hash::make('password123'): Enkripsi password wajib Laravel. Jangan pernah simpan password teks polos!
> - $this->call([FoodSeeder::class]): Memastikan data makanan langsung masuk ke database bersamaan dengan pembuatan akun admin.

[1.11] Terminal Command (Eksekusi Migrasi & Seeding)

Jalankan migrasi fresh beserta seeder ke database MySQL:

📄 Kondisi Awal Bawaan Generator Laravel — Default File

```
# Belum ada tabel terisi
```

🖍️ KODE YANG DITAMBAHKAN / DIKETIK PESERTA — Wajib Ditambahkan

```
php artisan migrate:fresh --seed
```

> 🔍 **Bedah Baris & Kenapa Ditulis Begitu?**
> - migrate:fresh akan drop semua tabel lama dan membuat ulang dari nol secara bersih.
> - Flag --seed langsung mengeksekusi DatabaseSeeder, sehingga akun admin (admin@gmail.com / password123) dan 5 menu makanan langsung siap diuji.

MODUL 2: BREEZE AUTH & CRUD MASTER MAKANAN (SISI ADMIN)

Menyiapkan sistem login admin dengan Laravel Breeze, upload file foto ke storage fisik, dan controller CRUD lengkap dengan Blade super simpel.

[2.1] Terminal Command (Instalasi Breeze & Symlink Storage)

Jalankan instalasi Breeze Blade dan link folder storage public:

📄 Kondisi Awal Bawaan Generator Laravel — Default File

```
# Breeze belum terpasang
```

🖍️ KODE YANG DITAMBAHKAN / DIKETIK PESERTA — Wajib Ditambahkan

```
composer require laravel/breeze --dev
php artisan breeze:install blade
# Opsi saat ditanya: Dark mode? -> NO | Testing framework? -> 0 (PHPUnit)

php artisan storage:link       // <-- [WAJIB] Membuat symlink foto public!
npm install && npm run build  // <-- [WAJIB] Build asset tampilan Breeze
```

🔍 **Bedah Baris & Kenapa Ditulis Begitu?**

- php artisan storage:link: SANGAT KRUSIAL! File upload disimpan di storage/app/public/. Perintah ini membuat folder shortcut public/storage. Tanpa ini, semua foto yang diupload akan 404 (gambar rusak).
- npm run build: Menghasilkan manifest asset Breeze di public/build. Jika error 'Vite manifest not found', pastikan perintah ini sudah dijalankan.

[2.2] app/Http/Controllers/FoodController.php

Buka FoodController.php. Isi ke-7 method resource standar untuk mengelola data menu makanan beserta upload dan hapus foto fisik.

📄 Kondisi Awal Bawaan Generator Laravel — Default File

```php
<?php

namespace App\Http\Controllers;

use App\Models\Food;
use Illuminate\Http\Request;

class FoodController extends Controller
{
    public function index() { // }
    public function create() { // }
    public function store(Request $request) { // }
    public function show(Food $food) { // }
    public function edit(Food $food) { // }
    public function update(Request $request, Food $food) { // }
    public function destroy(Food $food) { // }
}
```

🖍️ KODE YANG DITAMBAHKAN / DIKETIK PESERTA — Wajib Ditambahkan

```php
<?php

namespace App\Http\Controllers;

use App\Models\Food;
use Illuminate\Http\Request;
use Illuminate\Support\Facades\Storage; // <-- [TAMBAHKAN] Wajib untuk kelola file storage

class FoodController extends Controller
{
    // 1. Tampilkan tabel daftar makanan (index)
    public function index()
    {
        $foods = Food::latest()->paginate(10);
        return view('admin.foods.index', compact('foods'));
    }
```

```php
    // 2. Tampilkan form tambah makanan baru (create)
    public function create()
    {
        return view('admin.foods.create');
    }

    // 3. Simpan data makanan baru + upload gambar (store)
    public function store(Request $request)
    {
        $request->validate([
            'name'        => 'required|string|max:255',
            'category'    => 'required|in:Makanan,Minuman,Cemilan',
            'price'       => 'required|numeric|min:0',
            'description' => 'required|string',
            'image'       => 'nullable|image|mimes:jpeg,png,jpg|max:2048',
        ]);

        $imagePath = null;
        if ($request->hasFile('image')) {
            $imagePath = $request->file('image')->store('foods', 'public');
        }

        Food::create([
            'name'        => $request->name,
            'category'    => $request->category,
            'price'       => $request->price,
            'description' => $request->description,
            'image'       => $imagePath,
        ]);

        return redirect()->route('foods.index')
            ->with('success', 'Makanan berhasil ditambahkan!');
    }

    // 4. Detail makanan (show -> redirect saja ke edit)
    public function show(Food $food)
    {
        return view('admin.foods.edit', compact('food'));
    }

    // 5. Tampilkan form edit makanan (edit)
    public function edit(Food $food)
    {
        return view('admin.foods.edit', compact('food'));
    }

    // 6. Update data makanan + ganti gambar lama (update)
    public function update(Request $request, Food $food)
    {
        $request->validate([
            'name'        => 'required|string|max:255',
            'category'    => 'required|in:Makanan,Minuman,Cemilan',
            'price'       => 'required|numeric|min:0',
            'description' => 'required|string',
            'image'       => 'nullable|image|mimes:jpeg,png,jpg|max:2048',
        ]);

        $imagePath = $food->image;
        if ($request->hasFile('image')) {
            // Hapus gambar lama jika ada di storage fisik
            if ($food->image && Storage::disk('public')->exists($food->image)) {
                Storage::disk('public')->delete($food->image);
            }
```

```
            // Simpan gambar baru
            $imagePath = $request->file('image')->store('foods', 'public');
        }

        $food->update([
            'name'        => $request->name,
            'category'    => $request->category,
            'price'       => $request->price,
            'description' => $request->description,
            'image'       => $imagePath,
        ]);

        return redirect()->route('foods.index')
            ->with('success', 'Makanan berhasil diperbarui!');
    }

    // 7. Hapus data makanan + hapus gambar fisik (destroy)
    public function destroy(Food $food)
    {
        if ($food->image && Storage::disk('public')->exists($food->image)) {
            Storage::disk('public')->delete($food->image);
        }
        $food->delete();

        return redirect()->route('foods.index')
            ->with('success', 'Makanan berhasil dihapus!');
    }
}
```

🔍 **Bedah Baris & Kenapa Ditulis Begitu?**

- store('foods', 'public'): Menyimpan gambar ke storage/app/public/foods/. Laravel otomatis memberi nama acak unik.
- Storage::disk('public')->delete(...): Mencegah penumpukan file sampah foto lama saat diedit atau dihapus.
- session('success'): Flash message notifikasi ke user setelah aksi selesai.

[2.3] resources/views/admin/foods/index.blade.php

Buat file baru di resources/views/admin/foods/index.blade.php. Tampilan tabel daftar makanan admin berbalut template card dan tabel bersih rapi bergaris.

📄 Kondisi Awal Bawaan Generator Laravel — Default File

```
# File belum ada, buat baru
```

🖍️ KODE YANG DITAMBAHKAN / DIKETIK PESERTA — Wajib Ditambahkan

```
<x-app-layout>
    <div class="py-12">
        <div class="max-w-7xl mx-auto sm:px-6 lg:px-8">
            <div class="bg-white p-6 rounded-lg shadow-sm">
                <div class="flex justify-between items-center mb-6">
                    <h2 class="text-xl font-bold">Daftar Menu Makanan</h2>
                    <a href="{{ route('foods.create') }}" class="px-4 py-2 bg-gray-800 text-white rounded
text-sm font-medium hover:bg-black">
                        + Tambah Menu Baru
                    </a>
                </div>

                @if(session('success'))
                    <div class="mb-4 p-3 bg-gray-100 border border-gray-400 rounded text-sm font-medium">
                        {{ session('success') }}
```

```
                    </div>
                @endif

                <div class="overflow-x-auto">
                    <table class="w-full border-collapse border border-gray-300 text-left">
                        <thead>
                            <tr class="bg-gray-100">
                                <th class="border border-gray-300 p-3">Foto</th>
                                <th class="border border-gray-300 p-3">Nama Menu</th>
                                <th class="border border-gray-300 p-3">Kategori</th>
                                <th class="border border-gray-300 p-3">Harga</th>
                                <th class="border border-gray-300 p-3">Aksi</th>
                            </tr>
                        </thead>
                        <tbody>
                            @forelse($foods as $food)
                            <tr class="hover:bg-gray-50">
                                <td class="border border-gray-300 p-3">
                                    @if($food->image)
                                        <img src="{{ asset('storage/' . $food->image) }}" class="h-12 w-12
object-cover rounded" alt="{{ $food->name }}">
                                    @else
                                        <span class="text-xs text-gray-400">-</span>
                                    @endif
                                </td>
                                <td class="border border-gray-300 p-3 font-medium">{{ $food->name }}</td>
                                <td class="border border-gray-300 p-3">{{ $food->category }}</td>
                                <td class="border border-gray-300 p-3">Rp {{ number_format($food->price, 0,
',', '.') }}</td>
                                <td class="border border-gray-300 p-3 space-x-2">
                                    <a href="{{ route('foods.edit', $food->id) }}" class="text-blue-600
underline text-sm">Edit</a>
                                    <form action="{{ route('foods.destroy', $food->id) }}" method="POST"
class="inline" onsubmit="return confirm('Yakin hapus data ini?')">
                                        @csrf
                                        @method('DELETE')
                                        <button type="submit" class="text-red-600 underline text-
sm">Hapus</button>
                                    </form>
                                </td>
                            </tr>
                            @empty
                            <tr>
                                <td colspan="5" class="border border-gray-300 p-4 text-center text-gray-
500">Belum ada data makanan.</td>
                            </tr>
                            @endforelse
                        </tbody>
                    </table>
                </div>

                <div class="mt-4">
                    {{ $foods->links() }}
                </div>
            </div>
        </div>
    </div>
</x-app-layout>
```

🔍 **Bedah Baris & Kenapa Ditulis Begitu?**

- <x-app-layout>: Menggunakan layout admin bawaan Breeze lengkap dengan header dan session login.
- Card wrapper (bg-white p-6 rounded-lg shadow-sm): Struktur card simpel dan rapi.
- Tabel Bergaris: w-full border-collapse border border-gray-300 membuat tabel jelas dan terstruktur.
- Tombol Gelap: px-4 py-2 bg-gray-800 text-white rounded untuk aksi utama, text-red-600 untuk hapus, text-blue-600 untuk edit.

[2.4] resources/views/admin/foods/create.blade.php

Buat file baru di resources/views/admin/foods/create.blade.php. Form tambah menu dengan input w-full border rounded p-2 yang bersih dan rapi.

📄 Kondisi Awal Bawaan Generator Laravel — Default File

```
# File belum ada, buat baru
```

🖍️ KODE YANG DITAMBAHKAN / DIKETIK PESERTA — Wajib Ditambahkan

```
<x-app-layout>
    <div class="py-12">
        <div class="max-w-2xl mx-auto sm:px-6 lg:px-8">
            <div class="bg-white p-6 rounded-lg shadow-sm">
                <div class="flex justify-between items-center mb-6">
                    <h2 class="text-xl font-bold">Tambah Makanan</h2>
                    <a href="{{ route('foods.index') }}" class="px-3 py-1 border rounded text-sm hover:bg-
gray-100">
                        &larr; Kembali
                    </a>
                </div>

                <form action="{{ route('foods.store') }}" method="POST" enctype="multipart/form-data">
                    @csrf

                    <div class="mb-4">
                        <label class="block font-medium mb-1">Nama Makanan</label>
                        <input type="text" name="name" value="{{ old('name') }}" class="w-full border rounded
p-2" required>
                        @error('name') <span class="text-red-500 text-sm">{{ $message }}</span> @enderror
                    </div>

                    <div class="mb-4">
                        <label class="block font-medium mb-1">Kategori</label>
                        <select name="category" class="w-full border rounded p-2" required>
                            <option value="">-- Pilih Kategori --</option>
                            <option value="Makanan" {{ old('category') == 'Makanan' ? 'selected' : ''
}}>Makanan</option>
                            <option value="Minuman" {{ old('category') == 'Minuman' ? 'selected' : ''
}}>Minuman</option>
                            <option value="Cemilan" {{ old('category') == 'Cemilan' ? 'selected' : ''
}}>Cemilan</option>
                        </select>
                        @error('category') <span class="text-red-500 text-sm">{{ $message }}</span> @enderror
                    </div>

                    <div class="mb-4">
                        <label class="block font-medium mb-1">Harga (Rp)</label>
                        <input type="number" name="price" value="{{ old('price') }}" min="0" class="w-full
border rounded p-2" required>
                        @error('price') <span class="text-red-500 text-sm">{{ $message }}</span> @enderror
                    </div>
```

```
                    <div class="mb-4">
                        <label class="block font-medium mb-1">Deskripsi</label>
                        <textarea name="description" class="w-full border rounded p-2" rows="3" required>{{
old('description') }}</textarea>
                        @error('description') <span class="text-red-500 text-sm">{{ $message }}</span>
@enderror
                    </div>

                    <div class="mb-4">
                        <label class="block font-medium mb-1">Gambar Makanan</label>
                        <input type="file" name="image" class="w-full border rounded p-2">
                        @error('image') <span class="text-red-500 text-sm">{{ $message }}</span> @enderror
                    </div>

                    <div class="mt-6 flex justify-end">
                        <button type="submit" class="px-5 py-2 bg-gray-800 text-white rounded font-medium
hover:bg-black">
                            Simpan Data
                        </button>
                    </div>
                </form>
            </div>
        </div>
    </div>
</x-app-layout>
```

🔍 **Bedah Baris & Kenapa Ditulis Begitu?**

- enctype="multipart/form-data": ATURAN WAJIB HTML! Tanpa atribut ini, binary gambar tidak akan dikirim browser ke server.
- w-full border rounded p-2: Rumus class input minimalis seragam yang mudah diingat.
- old('name') & old('category'): Menjaga data yang sudah diketik tidak hilang jika validasi form gagal.

[2.5] resources/views/admin/foods/edit.blade.php

Buat file baru di resources/views/admin/foods/edit.blade.php. Form edit menu dengan preview foto saat ini dan method spoofing PUT.

📄 Kondisi Awal Bawaan Generator Laravel — Default File

```
# File belum ada, buat baru
```

🖍️ KODE YANG DITAMBAHKAN / DIKETIK PESERTA — Wajib Ditambahkan

```
<x-app-layout>
    <div class="py-12">
        <div class="max-w-2xl mx-auto sm:px-6 lg:px-8">
            <div class="bg-white p-6 rounded-lg shadow-sm">
                <div class="flex justify-between items-center mb-6">
                    <h2 class="text-xl font-bold">Edit Makanan</h2>
                    <a href="{{ route('foods.index') }}" class="px-3 py-1 border rounded text-sm hover:bg-
gray-100">
                        &larr; Kembali
                    </a>
                </div>

                <form action="{{ route('foods.update', $food->id) }}" method="POST" enctype="multipart/form-
data">
                    @csrf
                    @method('PUT')

                    <div class="mb-4">
```

```
                        <label class="block font-medium mb-1">Nama Makanan</label>
                        <input type="text" name="name" value="{{ old('name', $food->name) }}" class="w-full
border rounded p-2" required>
                        @error('name') <span class="text-red-500 text-sm">{{ $message }}</span> @enderror
                    </div>

                    <div class="mb-4">
                        <label class="block font-medium mb-1">Kategori</label>
                        <select name="category" class="w-full border rounded p-2" required>
                            <option value="Makanan" {{ old('category', $food->category) == 'Makanan' ?
'selected' : '' }}>Makanan</option>
                            <option value="Minuman" {{ old('category', $food->category) == 'Minuman' ?
'selected' : '' }}>Minuman</option>
                            <option value="Cemilan" {{ old('category', $food->category) == 'Cemilan' ?
'selected' : '' }}>Cemilan</option>
                        </select>
                        @error('category') <span class="text-red-500 text-sm">{{ $message }}</span> @enderror
                    </div>

                    <div class="mb-4">
                        <label class="block font-medium mb-1">Harga (Rp)</label>
                        <input type="number" name="price" value="{{ old('price', $food->price) }}" min="0"
class="w-full border rounded p-2" required>
                        @error('price') <span class="text-red-500 text-sm">{{ $message }}</span> @enderror
                    </div>

                    <div class="mb-4">
                        <label class="block font-medium mb-1">Deskripsi</label>
                        <textarea name="description" class="w-full border rounded p-2" rows="3" required>{{
old('description', $food->description) }}</textarea>
                        @error('description') <span class="text-red-500 text-sm">{{ $message }}</span>
@enderror
                    </div>

                    <div class="mb-4">
                        <label class="block font-medium mb-1">Gambar Baru (Kosongkan jika tidak diganti)
</label>
                        @if($food->image)
                            <div class="mb-2">
                                <img src="{{ asset('storage/' . $food->image) }}" class="h-20 w-20 object-
cover rounded" alt="{{ $food->name }}">
                            </div>
                        @endif
                        <input type="file" name="image" class="w-full border rounded p-2">
                        @error('image') <span class="text-red-500 text-sm">{{ $message }}</span> @enderror
                    </div>

                    <div class="mt-6 flex justify-end">
                        <button type="submit" class="px-5 py-2 bg-gray-800 text-white rounded font-medium
hover:bg-black">
                            Update Data
                        </button>
                    </div>
                </form>
            </div>
        </div>
    </div>
</x-app-layout>
```

> 🔍 **Bedah Baris & Kenapa Ditulis Begitu?**
> - @method('PUT'): Menyisipkan spoofing HTTP method PUT yang diwajibkan oleh route update.
> - old('name', $food->name): Nilai default mengambil dari database lama, namun otomatis diperbarui jika input form gagal validasi.

MODUL 3: SISI CUSTOMER (KATALOG MENU & TRANSAKSI CHECKOUT)

Menampilkan katalog menu untuk pembeli, input nomor meja, filter kategori dengan JavaScript 3 baris, serta transaksi multi-tabel aman dengan DB::transaction.

[3.1] app/Http/Controllers/OrderController.php (Bagian Transaksi Customer)

Buka OrderController.php. Isi method index() untuk menampilkan katalog publik, dan store() untuk memproses transaksi pesanan multi-tabel.

📄 Kondisi Awal Bawaan Generator Laravel — Default File

```php
<?php

namespace App\Http\Controllers;

use App\Models\Order;
use Illuminate\Http\Request;

class OrderController extends Controller
{
    public function index() { // }
    public function create() { // }
    public function store(Request $request) { // }
    public function show(Order $order) { // }
    public function edit(Order $order) { // }
    public function update(Request $request, Order $order) { // }
    public function destroy(Order $order) { // }
}
```

🖍️ KODE YANG DITAMBAHKAN / DIKETIK PESERTA — Wajib Ditambahkan

```php
<?php

namespace App\Http\Controllers;

use App\Models\Food;               // <-- [TAMBAHKAN] Model Menu
use App\Models\Order;              // <-- [TAMBAHKAN] Model Order Header
use App\Models\OrderDetail;        // <-- [TAMBAHKAN] Model Detail Order
use Illuminate\Http\Request;
use Illuminate\Support\Facades\DB; // <-- [TAMBAHKAN] Wajib untuk DB::transaction

class OrderController extends Controller
{
    // 1. Tampilkan katalog menu publik untuk customer
    public function index()
    {
        $foods = Food::all();
        return view('customer.index', compact('foods'));
    }

    // 2. Simpan transaksi pesanan (multi-tabel aman dengan DB::transaction)
    public function store(Request $request)
    {
```

```php
        $request->validate([
            'customer_name' => 'required|string|max:255',
            'table_number'  => 'required|integer|min:1',
            'items'         => 'required|array',
            'items.*'       => 'nullable|integer|min:0',
        ]);

        // Filter: Hanya ambil menu yang jumlah porsinya > 0
        $orderedItems = array_filter($request->items, fn($qty) => (int)$qty > 0);

        if (empty($orderedItems)) {
            return back()->with('error', 'Pilih minimal satu menu dengan jumlah porsi lebih dari 0!');
        }

        DB::beginTransaction();
        try {
            // A. Simpan data header pesanan ke tabel 'orders'
            $order = Order::create([
                'customer_name' => $request->customer_name,
                'table_number'  => $request->table_number,
                'total_price'   => 0, // Akan dihitung dan diupdate setelah rincian dihitung
                'status'        => 'Pending',
            ]);

            $totalPrice = 0;

            // B. Looping simpan rincian per menu ke tabel 'order_details'
            foreach ($orderedItems as $foodId => $quantity) {
                $food = Food::findOrFail($foodId);
                $subtotal = $food->price * $quantity;
                $totalPrice += $subtotal;

                OrderDetail::create([
                    'order_id' => $order->id,
                    'food_id'  => $food->id,
                    'quantity' => $quantity,
                    'subtotal' => $subtotal,
                ]);
            }

            // C. Perbarui total harga akhir di tabel header 'orders'
            $order->update(['total_price' => $totalPrice]);

            DB::commit(); // Transaksi berhasil, kunci data permanen ke MySQL

            return redirect()->route('customer.index')
                ->with('success', "Pesanan Meja {$order->table_number} atas nama {$order->customer_name}
berhasil! Total: Rp " . number_format($totalPrice));

        } catch (\Exception $e) {
            DB::rollBack(); // Terjadi error, batalkan semua insert data!
            return back()->with('error', 'Gagal memproses pesanan: ' . $e->getMessage());
        }
    }
}
```

🔍 **Bedah Baris & Kenapa Ditulis Begitu?**

- array_filter($request->items, fn($qty) => (int)$qty > 0): Menyaring item yang porsinya 0 sehingga database tidak menyimpan baris kosong.
- DB::beginTransaction() & DB::rollBack(): PRINSIP ACID DATABASE! Menjamin konsistensi: jika insert ke order_details gagal, pembuatan header di orders otomatis dibatalkan. Tidak ada 'data hantu' di database.
- $food->price * $quantity: Harga selalu dihitung dari tabel master foods di server, BUKAN dari kiriman input form, sehingga mencegah manipulasi harga oleh hacker.

[3.2] resources/views/customer/index.blade.php

Buat file baru di resources/views/customer/index.blade.php. Tampilan katalog pelanggan publik rapi, bersih, card grid terstruktur, dan modal konfirmasi.

📄 Kondisi Awal Bawaan Generator Laravel — Default File

```
# File belum ada, buat baru
```

🖍️ KODE YANG DITAMBAHKAN / DIKETIK PESERTA — Wajib Ditambahkan

```
<!DOCTYPE html>
<html lang="id">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>Pemesanan Makanan - Menu Pelanggan</title>
    <script src="https://cdn.tailwindcss.com"></script>
</head>
<body class="bg-gray-50 min-h-screen py-8">
    <div class="max-w-5xl mx-auto px-4 sm:px-6 lg:px-8">

        {{-- Top Bar Navigasi Admin / Publik --}}
        <div class="flex justify-between items-center mb-6 pt-2">
            <span class="text-xs font-semibold uppercase text-gray-500">Pemesanan Mandiri</span>
            <a href="{{ route('dashboard') }}" class="text-xs font-semibold text-gray-700 bg-white border
border-gray-300 px-3 py-1.5 rounded hover:bg-gray-100">
                Portal Admin &rarr;
            </a>
        </div>

        {{-- Header Katalog --}}
        <div class="text-center mb-8">
            <h1 class="text-2xl font-bold">Daftar Menu Restoran</h1>
            <p class="text-gray-600 text-sm mt-1">Sistem Pemesanan Mandiri Meja Makan</p>
        </div>

        {{-- Notifikasi Flash Sukses / Gagal --}}
        @if(session('success'))
            <div class="mb-4 p-3 bg-gray-100 border border-gray-400 text-gray-800 text-sm rounded font-medium
text-center">
                {{ session('success') }}
            </div>
        @endif

        @if(session('error'))
            <div class="mb-4 p-3 bg-red-50 border border-red-300 text-red-700 text-sm rounded font-medium
text-center">
                {{ session('error') }}
            </div>
        @endif
```

```
        @if($errors->any())
            <div class="mb-6 p-4 bg-red-50 border border-red-300 text-red-700 text-sm rounded">
                <p class="font-semibold mb-1">Periksa kembali data Anda:</p>
                <ul class="list-disc list-inside">
                    @foreach($errors->all() as $error)
                        <li>{{ $error }}</li>
                    @endforeach
                </ul>
            </div>
        @endif

        <form id="orderForm" action="{{ route('customer.checkout') }}" method="POST">
            @csrf

            {{-- 1. Card Data Pemesan --}}
            <div class="bg-white p-6 rounded-lg border shadow-sm mb-6">
                <h2 class="text-lg font-bold mb-4 pb-2 border-b">1. Data Pemesan</h2>
                <div class="grid grid-cols-1 md:grid-cols-2 gap-4">
                    <div>
                        <label for="customer_name" class="block font-medium text-sm mb-1">Nama Pemesan</label>
                        <input type="text" id="customer_name" name="customer_name" value="{{
old('customer_name') }}" required
                               placeholder="Contoh: Budi Santoso"
                               class="w-full border rounded p-2">
                    </div>
                    <div>
                        <label for="table_number" class="block font-medium text-sm mb-1">Nomor Meja</label>
                        <input type="number" id="table_number" name="table_number" value="{{
old('table_number') }}" min="1" required
                               placeholder="Contoh: 5"
                               class="w-full border rounded p-2">
                    </div>
                </div>
            </div>

            {{-- 2. Tombol Filter Kategori (Pill) --}}
            <div class="flex justify-center gap-2 mb-6">
                <button type="button" onclick="filterMenu('all', this)" class="btn-category px-4 py-1.5
rounded border text-sm font-medium bg-gray-800 text-white">
                    Semua
                </button>
                <button type="button" onclick="filterMenu('Makanan', this)" class="btn-category px-4 py-1.5
rounded border text-sm font-medium bg-white hover:bg-gray-100">
                    Makanan
                </button>
                <button type="button" onclick="filterMenu('Minuman', this)" class="btn-category px-4 py-1.5
rounded border text-sm font-medium bg-white hover:bg-gray-100">
                    Minuman
                </button>
                <button type="button" onclick="filterMenu('Cemilan', this)" class="btn-category px-4 py-1.5
rounded border text-sm font-medium bg-white hover:bg-gray-100">
                    Cemilan
                </button>
            </div>

            {{-- 3. Grid Daftar Menu Makanan --}}
            <div class="grid grid-cols-1 md:grid-cols-3 gap-4 mb-6">
                @forelse($foods as $food)
                <div class="menu-item bg-white border rounded-lg shadow-sm overflow-hidden flex flex-col
justify-between p-4" data-category="{{ $food->category }}">
                    <div>
                        @if($food->image)
                            <img src="{{ asset('storage/' . $food->image) }}" class="w-full h-36 object-cover
```

```
rounded mb-3">
                        @else
                            <div class="w-full h-36 bg-gray-100 rounded flex items-center justify-center text-
gray-400 text-xs mb-3">
                                Tanpa Gambar
                            </div>
                        @endif
                        <div class="flex justify-between items-center mb-1">
                            <span class="px-2 py-0.5 rounded text-xs border text-gray-700">
                                {{ $food->category }}
                            </span>
                            <span class="font-bold">
                                Rp {{ number_format($food->price, 0, ',', '.') }}
                            </span>
                        </div>
                        <h3 class="font-bold text-base mt-2">{{ $food->name }}</h3>
                        <p class="text-xs text-gray-600 mt-1 line-clamp-2">{{ $food->description }}</p>
                    </div>
                    <div class="mt-4 pt-3 border-t flex items-center justify-between">
                        <label class="text-xs font-medium">Porsi:</label>
                        <input type="number" name="items[{{ $food->id }}]" min="0" value="{{ old('items.' .
$food->id, 0) }}"
                               data-name="{{ $food->name }}" data-price="{{ $food->price }}"
                               class="item-qty w-20 text-center border rounded p-1 text-sm font-bold">
                    </div>
                </div>
                @empty
                <div class="col-span-3 text-center py-8 text-gray-400">
                    Menu belum tersedia.
                </div>
                @endforelse
            </div>

            {{-- 4. Tombol Submit Pesanan --}}
            <div class="text-right mb-12">
                <button type="button" onclick="confirmCheckout()" class="px-6 py-2.5 bg-gray-800 text-white
rounded font-medium hover:bg-black shadow-sm">
                    Kirim Pesanan Sekarang &rarr;
                </button>
            </div>

            {{-- 5. Modal Konfirmasi Pesanan Interaktif --}}
            <div id="confirmModal" class="fixed inset-0 bg-black/50 hidden items-center justify-center p-4 z-
50">
                <div class="bg-white p-6 rounded-lg max-w-md w-full border shadow-lg">
                    <h3 class="text-lg font-bold mb-3 border-b pb-2">Konfirmasi Pesanan</h3>
                    <div class="text-sm text-gray-700 mb-3 space-y-1">
                        <div>Nama Pemesan: <strong id="modalName"></strong></div>
                        <div>Nomor Meja: <strong id="modalTable"></strong></div>
                    </div>
                    <div class="border-t border-b py-3 mb-3 max-h-48 overflow-y-auto">
                        <ul id="modalItemList" class="text-sm space-y-1"></ul>
                    </div>
                    <div class="flex justify-between font-bold text-base mb-6">
                        <span>Total Bayar:</span>
                        <span id="modalTotalPrice">Rp 0</span>
                    </div>
                    <div class="flex gap-3">
                        <button type="button" onclick="closeConfirmationModal()" class="w-1/2 px-4 py-2 border
rounded text-sm font-medium hover:bg-gray-100">
                            Batal
                        </button>
                        <button type="submit" class="w-1/2 px-4 py-2 bg-gray-800 text-white rounded text-sm
```

```
font-medium hover:bg-black">
                            Ya, Kirim
                        </button>
                    </div>
                </div>
            </div>
        </form>

    </div>

    {{-- JAVASCRIPT FILTER & MODAL KONFIRMASI --}}
    <script>
        // 1. Filter Kategori Menu
        function filterMenu(kategori, btn) {
            if (btn) {
                document.querySelectorAll('.btn-category').forEach(b => {
                    b.className = 'btn-category px-4 py-1.5 rounded border text-sm font-medium bg-white
hover:bg-gray-100';
                });
                btn.className = 'btn-category px-4 py-1.5 rounded border text-sm font-medium bg-gray-800 text-
white';
            }
            document.querySelectorAll('.menu-item').forEach(card => {
                const match = (kategori === 'all' || card.dataset.category === kategori);
                card.style.display = match ? 'flex' : 'none';
            });
        }

        // 2. Modal Konfirmasi Pesanan
        function confirmCheckout() {
            const nama = document.getElementById('customer_name').value.trim();
            const meja = document.getElementById('table_number').value.trim();
            if (!nama || !meja || parseInt(meja) < 1) {
                alert('Nama pemesan dan nomor meja (minimal 1) wajib diisi!');
                return false;
            }

            let total = 0, count = 0;
            const listEl = document.getElementById('modalItemList');
            listEl.innerHTML = '';

            document.querySelectorAll('.item-qty').forEach(input => {
                const qty = parseInt(input.value) || 0;
                if (qty > 0) {
                    count++;
                    const subtotal = qty * parseFloat(input.dataset.price);
                    total += subtotal;

                    const li = document.createElement('li');
                    li.className = 'flex justify-between py-1 border-b border-gray-100';

                    const s1 = document.createElement('span');
                    s1.textContent = input.dataset.name + ' x' + qty;

                    const s2 = document.createElement('span');
                    s2.className = 'font-bold';
                    s2.textContent = 'Rp ' + subtotal.toLocaleString('id-ID');

                    li.appendChild(s1);
                    li.appendChild(s2);
                    listEl.appendChild(li);
                }
            });
```

```
            if (count === 0) {
                alert('Pilih minimal satu menu dengan porsi lebih dari 0!');
                return false;
            }

            document.getElementById('modalName').textContent = nama;
            document.getElementById('modalTable').textContent = meja;
            document.getElementById('modalTotalPrice').textContent = 'Rp ' + total.toLocaleString('id-ID');

            const modal = document.getElementById('confirmModal');
            modal.classList.remove('hidden');
            modal.classList.add('flex');
        }

        function closeConfirmationModal() {
            const modal = document.getElementById('confirmModal');
            modal.classList.remove('flex');
            modal.classList.add('hidden');
        }
    </script>
</body>
</html>
```

🔍 **Bedah Baris & Kenapa Ditulis Begitu?**

- Standalone Page: Halaman katalog pelanggan tidak memakai <x-app-layout> karena terbuka untuk umum tanpa login admin.
- CDN Tailwind: Memanggil cdn.tailwindcss.com sehingga styling modern langsung aktif secara mandiri.
- @if($errors->any()): Sangat vital! Menampilkan card error jika data input pelanggan gagal divalidasi controller.
- Modal Interaktif: Menggunakan Tailwind fixed inset-0 bg-black/50 yang membedah rincian pesanan dan total harga secara rapi sebelum disubmit ke server.
- old('items.' . $food->id, 0): Menjaga porsi pilihan pelanggan tidak hilang jika terjadi error validasi nama/meja.

MODUL 4: SISI ADMIN (REKAP TRANSAKSI, UPDATE STATUS & ROUTING)

Menampilkan rekapitulasi transaksi masuk di dashboard admin, mengubah status transaksi secara instan, dan mengintegrasikan seluruh routing di routes/web.php.

[4.1] app/Http/Controllers/OrderController.php (Tambahan Method Admin)

Buka OrderController.php kembali. Tambahkan 2 method berikut di bagian bawah class untuk keperluan dashboard admin:

📄 Kondisi Awal Bawaan Generator Laravel — Default File

```
// Isi OrderController.php dari Modul 3 sebelumnya
// Hanya berisi index() dan store()
```

🖍️ KODE YANG DITAMBAHKAN / DIKETIK PESERTA — Wajib Ditambahkan

```
    // 3. <-- [TAMBAHKAN] Dashboard rekap pesanan admin (Eager Loading with)
    public function adminDashboard()
    {
        $orders = Order::with('orderDetails.food')->latest()->get();
        return view('dashboard', compact('orders'));
    }

    // 4. <-- [TAMBAHKAN] Update status pesanan (Pending -> Diproses -> Selesai)
    public function updateStatus(Request $request, $id)
    {
```

```
        $request->validate(['status' => 'required|in:Pending,Diproses,Selesai']);

        $order = Order::findOrFail($id);
        $order->update(['status' => $request->status]);

        return back()->with('success', "Status pesanan #{$order->id} berhasil diubah menjadi {$order->status}!");
    }
} // <-- Tutup class OrderController
```

🔍 **Bedah Baris & Kenapa Ditulis Begitu?**

- Order::with('orderDetails.food'): TEKNIK EAGER LOADING! Mengambil data pesanan sekaligus rincian dan nama menu makanannya dalam query teroptimasi. Menghindari masalah N+1 Query yang sering ditanyakan asesor BNSP.
- updateStatus(): Menerima request HTTP PATCH dari form mini di dashboard dan langsung memperbarui status pesanan.

[4.2] resources/views/dashboard.blade.php

Buka resources/views/dashboard.blade.php. Gantikan isi default bawaan Breeze dengan tabel rekap pesanan, badge status rapi, dan select update status instan.

📄 Kondisi Awal Bawaan Generator Laravel — Default File

```
<x-app-layout>
    <x-slot name="header">
        <h2 class="font-semibold text-xl text-gray-800 leading-tight">
            {{ __('Dashboard') }}
        </h2>
    </x-slot>

    <div class="py-12">
        <div class="max-w-7xl mx-auto sm:px-6 lg:px-8">
            <div class="bg-white overflow-hidden shadow-sm sm:rounded-lg">
                <div class="p-6 text-gray-900">
                    {{ __("You're logged in!") }}
                </div>
            </div>
        </div>
    </div>
</x-app-layout>
```

🖍️ KODE YANG DITAMBAHKAN / DIKETIK PESERTA — Wajib Ditambahkan

```
<x-app-layout>
    <div class="py-12">
        <div class="max-w-7xl mx-auto sm:px-6 lg:px-8">
            <div class="bg-white p-6 rounded-lg shadow-sm">
                <div class="flex justify-between items-center mb-6">
                    <h2 class="text-xl font-bold">Daftar Pesanan Masuk</h2>
                    <div class="flex gap-2">
                        <a href="{{ route('foods.index') }}" class="px-4 py-2 border rounded text-sm font-medium hover:bg-gray-100">
                            Kelola Menu
                        </a>
                        <a href="{{ route('customer.index') }}" target="_blank" class="px-4 py-2 bg-gray-800 text-white rounded text-sm font-medium hover:bg-black">
                            Lihat Menu Customer &rarr;
                        </a>
                    </div>
                </div>
```

```
                @if(session('success'))
                    <div class="mb-4 p-3 bg-gray-100 border border-gray-400 rounded text-sm font-medium">
                        {{ session('success') }}
                    </div>
                @endif

                <div class="overflow-x-auto">
                    <table class="w-full border-collapse border border-gray-300 text-left">
                        <thead>
                            <tr class="bg-gray-100">
                                <th class="border border-gray-300 p-3"># ID</th>
                                <th class="border border-gray-300 p-3">Pelanggan</th>
                                <th class="border border-gray-300 p-3">No. Meja</th>
                                <th class="border border-gray-300 p-3">Rincian Pesanan</th>
                                <th class="border border-gray-300 p-3">Total</th>
                                <th class="border border-gray-300 p-3">Status</th>
                                <th class="border border-gray-300 p-3 text-center">Ubah Status</th>
                            </tr>
                        </thead>
                        <tbody>
                            @forelse($orders as $order)
                            <tr class="hover:bg-gray-50">
                                <td class="border border-gray-300 p-3 font-bold">#{{ $order->id }}</td>
                                <td class="border border-gray-300 p-3">{{ $order->customer_name }}</td>
                                <td class="border border-gray-300 p-3">Meja {{ $order->table_number }}</td>
                                <td class="border border-gray-300 p-3">
                                    <ul class="list-disc list-inside text-xs">
                                        @foreach($order->orderDetails as $detail)
                                            <li>{{ $detail->food->name ?? 'Menu' }} x{{ $detail->quantity }}
(Rp {{ number_format($detail->subtotal, 0, ',', '.') }})</li>
                                        @endforeach
                                    </ul>
                                </td>
                                <td class="border border-gray-300 p-3 font-bold">Rp {{ number_format($order-
>total_price, 0, ',', '.') }}</td>
                                <td class="border border-gray-300 p-3">
                                    <span class="border px-2 py-0.5 rounded text-xs font-semibold">
                                        {{ $order->status }}
                                    </span>
                                </td>
                                <td class="border border-gray-300 p-3 text-center">
                                    <form action="{{ route('admin.orders.updateStatus', $order->id) }}"
method="POST">
                                        @csrf
                                        @method('PATCH')
                                        <select name="status" onchange="this.form.submit()" class="text-xs
border rounded p-1">
                                            <option value="Pending" {{ $order->status == 'Pending' ?
'selected' : '' }}>Pending</option>
                                            <option value="Diproses" {{ $order->status == 'Diproses' ?
'selected' : '' }}>Diproses</option>
                                            <option value="Selesai" {{ $order->status == 'Selesai' ?
'selected' : '' }}>Selesai</option>
                                        </select>
                                    </form>
                                </td>
                            </tr>
                            @empty
                            <tr>
                                <td colspan="7" class="border border-gray-300 p-4 text-center text-gray-
500">Belum ada pesanan masuk.</td>
                            </tr>
                            @endforelse
```

```
                        </tbody>
                    </table>
                </div>
            </div>
        </div>
    </div>
</x-app-layout>
```

🔍 **Bedah Baris & Kenapa Ditulis Begitu?**

- w-full border-collapse border border-gray-300: Tabel bergaris rapi yang mudah diaudit asesor.
- Select status otomatis submit: onchange="this.form.submit()" membuat perubahan status langsung disimpan tanpa tombol tambahan.

[4.3] routes/web.php

Buka routes/web.php. Hubungkan seluruh routing aplikasi dari Modul 1 sampai Modul 4 secara terintegrasi.

📄 Kondisi Awal Bawaan Generator Laravel — Default File

```php
<?php

use App\Http\Controllers\ProfileController;
use Illuminate\Support\Facades\Route;

Route::get('/', function () {
    return view('welcome');
});

Route::get('/dashboard', function () {
    return view('dashboard');
})->middleware(['auth', 'verified'])->name('dashboard');

Route::middleware('auth')->group(function () {
    Route::get('/profile', [ProfileController::class, 'edit'])->name('profile.edit');
    Route::patch('/profile', [ProfileController::class, 'update'])->name('profile.update');
    Route::delete('/profile', [ProfileController::class, 'destroy'])->name('profile.destroy');
});

require __DIR__.'/auth.php';
```

🖍️ KODE YANG DITAMBAHKAN / DIKETIK PESERTA — Wajib Ditambahkan

```php
<?php

use App\Http\Controllers\ProfileController;
use App\Http\Controllers\FoodController;   // <-- [TAMBAHKAN] Import Controller Makanan
use App\Http\Controllers\OrderController;  // <-- [TAMBAHKAN] Import Controller Pesanan
use Illuminate\Support\Facades\Route;

// 1. SISI CUSTOMER (Rute Publik Tanpa Login)
Route::get('/', [OrderController::class, 'index'])->name('customer.index');          // <-- [UBAH] Arahkan ke katalog
Route::post('/checkout', [OrderController::class, 'store'])->name('customer.checkout'); // <-- [TAMBAHKAN] Proses pesanan

// 2. DASHBOARD REKAP ADMIN (Wajib Login Breeze)
Route::get('/dashboard', [OrderController::class, 'adminDashboard'])                 // <-- [UBAH] Panggil method adminDashboard
    ->middleware(['auth', 'verified'])
    ->name('dashboard');
```

```php
// 3. FITUR ADMIN TERPROTEKSI (Wajib Login)
Route::middleware('auth')->group(function () {
    // Breeze Profile Bawaan
    Route::get('/profile', [ProfileController::class, 'edit'])->name('profile.edit');
    Route::patch('/profile', [ProfileController::class, 'update'])->name('profile.update');
    Route::delete('/profile', [ProfileController::class, 'destroy'])->name('profile.destroy');

    // Update Status Pesanan (PATCH)
    Route::patch('/admin/orders/{id}/status', [OrderController::class, 'updateStatus']) // <-- [TAMBAHKAN]
        ->name('admin.orders.updateStatus');

    // CRUD Master Data Makanan (WAJIB DIBERI ALIAS ->names('foods'))
    Route::resource('/admin/foods', FoodController::class)->names('foods');             // <-- [TAMBAHKAN]
});

require __DIR__.'/auth.php';
```

🔍 Bedah Baris & Kenapa Ditulis Begitu?

- Route::resource('/admin/foods', ...)->names('foods'): SANGAT KRUSIAL! URL prefix adalah /admin/foods. Jika tanpa ->names('foods'), Laravel akan menamainya admin.foods.index. Karena view memanggil route('foods.index'), tanpa ->names('foods') aplikasi akan melempar fatal error: Route [foods.index] not defined.
- Route::middleware('auth')->group(...): Menjaga rute CRUD admin dan update status agar hanya bisa dibuka setelah login.

🎯 TIPS & TRIK HAFALAN & STRATEGI UJIAN SERKOM (21 OKTOBER)

📌 Mental Map 5 Langkah Inti Pengerjaan (Anti-Blank)

Selalu ikuti urutan: (1) Database & Migration -> (2) Model & Relasi -> (3) Breeze & Storage Link -> (4) Controller & Route -> (5) View Blade. JANGAN PERNAH membuat View sebelum Controller dan Tabel siap di database, karena akan memicu fatal error 'Class Not Found' atau 'Undefined Variable'.

📌 Contekan Praktis Breeze Styling (Di Mana Mencari Class di Proyek?)

Jika lupa nama class Tailwind saat ujian, contek langsung dari file bawaan Breeze:
- Card wrapper: Buka resources/views/dashboard.blade.php (py-12, max-w-7xl mx-auto sm:px-6 lg:px-8, bg-white shadow-sm sm:rounded-lg p-6).
- Form input & label: Buka resources/views/auth/login.blade.php (border-gray-300 focus:border-indigo-500 rounded-md shadow-sm w-full).
- Tombol aksi: Buka resources/views/components/ (Primary: bg-gray-800 text-white rounded-md, Danger: bg-red-600 text-white rounded-md).
- Tabel rapi: Gunakan min-w-full divide-y divide-gray-200 dengan header bg-gray-50 text-gray-500 uppercase.

📌 Rumus Hafalan Cepat 7 Method Controller CRUD

Ingat pola '2 Pasang + 3 Tunggal':
- Pasang 1: create() (buka form kosong) & store() (validasi + simpan ke DB)
- Pasang 2: edit() (buka form data lama) & update() (validasi + simpan perubahan)
- Tunggal 1: index() (ambil data + kirim ke tabel view)
- Tunggal 2: destroy() (hapus data + hapus foto fisik)
- Tunggal 3: show() (redirect ke edit atau detail)

📌 Rumus 3 Baris Sakti Upload Gambar

1. Cek: if ($request->hasFile('image'))
2. Simpan: $path = $request->file('image')->store('foods', 'public');
3. Hapus foto lama: if ($food->image && Storage::disk('public')->exists($food->image)) { Storage::disk('public')->delete($food->image); }

📌 **Rumus Anti-Lupa Relasi Database**

- Model Order (Induk): public function orderDetails() { return $this->hasMany(OrderDetail::class, 'order_id'); }
- Model OrderDetail (Anak): public function order() { return $this->belongsTo(Order::class, 'order_id'); }
- Relasi ke Makanan: public function food() { return $this->belongsTo(Food::class, 'food_id'); }

📌 **Rumus Transaksi Multi-Tabel (ACID)**

```
DB::beginTransaction();
try {
$order = Order::create([...]);
foreach ($items as $id => $qty) { OrderDetail::create([...]); }
$order->update(['total_price' => $total]);
DB::commit();
} catch (\Exception $e) {
DB::rollBack();
}
```

📌 **Checklist Pengujian oleh Asesor BNSP / LSP (Simulasi Uji Mandiri)**

Sebelum memanggil asesor untuk dinilai, uji sendiri 7 skenario berikut:
1. Buka http://localhost:8000/ : Katalog menu tampil & filter kategori berfungsi.
2. Isi nama, nomor meja, pilih porsi menu : Checkout berhasil & total harga benar.
3. Login admin di /login (admin@gmail.com / password123) : Masuk dashboard, pesanan tadi muncul.
4. Ubah status pesanan jadi 'Diproses' atau 'Selesai' : Status berhasil terupdate.
5. Buka Kelola Menu (/admin/foods) : Tambah menu baru dengan upload foto.
6. Edit menu & ganti foto baru : Foto lama terhapus dari storage, foto baru muncul.
7. Hapus menu : Data terhapus dan foto fisik ikut terhapus.

🔥 SESI GRILL-ME: 10 PERTANYAAN WAWANCARA ASESOR BNSP BESERTA JAWABAN JITU

Asesor Sertifikasi Kompetensi (BNSP/LSP) selalu melakukan tes wawancara lisan (oral assessment) untuk memastikan Anda benar-benar paham logika di balik kode yang Anda tulis. Pelajari 10 jawaban jitu di bawah ini:

❓ **Pertanyaan 1: Mengapa Anda menggunakan DB::beginTransaction() dan DB::rollBack() pada proses checkout pesanan?**

Jawaban Jitu: Untuk menjamin prinsip ACID (khususnya Atomicity). Karena checkout melibatkan penyimpanan ke dua tabel yang saling bergantung ('orders' dan 'order_details'), jika penyimpanan rincian menu gagal di tengah jalan (misal server error atau foreign key gagal), DB::rollBack() akan membatalkan seluruh perubahan. Ini mencegah timbulnya 'data hantu' (header order tersimpan tanpa ada detail rincian menu) dan memastikan integritas data keuangan tetap akurat.

❓ **Pertanyaan 2: Apa fungsi perintah 'php artisan storage:link' dan apa dampaknya jika lupa dijalankan?**

Jawaban Jitu: Secara default, Laravel menyimpan file upload di folder internal 'storage/app/public/' yang tidak bisa diakses langsung oleh browser demi alasan keamanan. Perintah 'storage:link' membuat symbolic link (shortcut) dari folder 'public/storage' ke 'storage/app/public/'. Jika lupa dijalankan, seluruh gambar yang diupload tidak akan tampil di browser (menghasilkan error HTTP 404 / gambar rusak).

❓ **Pertanyaan 3: Mengapa Anda menggunakan 'protected $guarded = ['id'];' daripada '$fillable' pada Model Eloquent?**

Jawaban Jitu: Ini adalah strategi efisiensi dan kecepatan saat ujian Serkom. '$fillable' bersifat whitelist di mana kita harus mendaftarkan setiap kolom satu per satu. Dengan '$guarded = ['id'];' (blacklist), kita melindungi kolom 'id' agar tidak dimanipulasi, sementara seluruh kolom lainnya otomatis diizinkan untuk mass-assignment (Food::create). Ini menghemat waktu pengetikan dan mencegah error MassAssignmentException jika ada penambahan kolom.

❓ Pertanyaan 4: Mengapa pada form edit Anda menggunakan direktif Blade @method('PUT'), padahal form HTML hanya mendukung GET dan POST?

Jawaban Jitu: Browser web secara native hanya mendukung method HTTP GET dan POST. Untuk mengikuti standar arsitektur RESTful API di mana operasi pembaruan data menggunakan method PUT atau PATCH, Laravel menyediakan fitur Method Spoofing. Direktif @method('PUT') menyisipkan tag tersembunyi <input type='hidden' name='_method' value='PUT'> sehingga router Laravel dapat mengenali request tersebut sebagai PUT.

❓ Pertanyaan 5: Apa fungsi direktif @csrf dan apa yang terjadi jika form tidak diberi @csrf?

Jawaban Jitu: @csrf menyisipkan token rahasia Cross-Site Request Forgery untuk memastikan bahwa request submit form benar-benar berasal dari aplikasi kita, bukan dari eksploitasi skrip pihak ketiga. Jika tidak dipasang, Laravel Middleware (VerifyCsrfToken) akan menolak kiriman form dengan melempar error '419 Page Expired'.

❓ Pertanyaan 6: Jelaskan relasi Eloquent antara model Food, Order, dan OrderDetail!

Jawaban Jitu: Relasi antara Order dan OrderDetail adalah One-to-Many: 1 transaksi Order memiliki banyak OrderDetail ($order->hasMany(OrderDetail::class)). Sebaliknya, OrderDetail memiliki relasi Many-to-One balik ke Order ($detail->belongsTo(Order::class)) dan Many-to-One ke Food ($detail->belongsTo(Food::class)). Dengan relasi ini, kita dapat memanggil nama menu yang dipesan di dashboard admin dengan sintaks elegan: $detail->food->name.

❓ Pertanyaan 7: Mengapa urutan pembuatan tabel pada migration harus: foods -> orders -> order_details?

Jawaban Jitu: Karena tabel 'order_details' memiliki dua foreign key ('order_id' yang merujuk ke tabel 'orders' dan 'food_id' yang merujuk ke tabel 'foods'). Di database relasional, tabel induk HARUS dibuat terlebih dahulu sebelum tabel anak. Jika tabel 'order_details' dijalankan lebih awal, MySQL akan menolak dengan error 'Foreign key constraint fails' karena tabel tujuan belum ada.

❓ Pertanyaan 8: Apa keuntungan menggunakan Eager Loading Order::with('orderDetails.food') di dashboard admin?

Jawaban Jitu: Untuk mencegah masalah 'N+1 Query Problem'. Jika kita hanya memanggil Order::all() (Lazy Loading), setiap kali loop menampilkan rincian dan nama makanan, Laravel akan mengeksekusi query database baru secara berulang-ulang. Dengan Eager Loading with('orderDetails.food'), Laravel hanya menjalankan 3 query teroptimasi di awal, sehingga loading dashboard jauh lebih cepat dan hemat memori server.

❓ Pertanyaan 9: Mengapa tag form tambah dan edit makanan wajib memiliki atribut enctype='multipart/form-data'?

Jawaban Jitu: Secara default, form HTML mengirimkan data dengan encoding teks 'application/x-www-form-urlencoded'. Atribut 'enctype="multipart/form-data"' memerintahkan browser untuk memecah data form menjadi bagian biner sehingga file fisik gambar dapat ditransmisikan ke server. Tanpa atribut ini, method $request->file('image') di controller akan selalu bernilai null.

❓ Pertanyaan 10: Bagaimana Anda memastikan tidak ada file foto sampah yang menumpuk di server saat makanan diedit atau dihapus?

Jawaban Jitu: Di FoodController pada method update() dan destroy(), kita melakukan pengecekan keberadaan foto lama dengan 'Storage::disk('public')->exists($food->image)'. Jika ada, kita menghapus file fisik tersebut dengan 'Storage::disk('public')->delete($food->image)' sebelum mengunggah foto baru atau sebelum record model dihapus.

🚨 KAMUS ERROR & TROUBLESHOOTING INSTAN (SOLUSI 10 DETIK)

Pesan Error / Gejala	Penyebab Utama	Solusi Instan 10 Detik
419 Page Expired	Lupa memasang token CSRF pada form HTML.	**Tambahkan @csrf tepat di bawah tag pembuka <form>.**
Route [foods.create] not defined	Rute resource /admin/foods tidak diberi alias nama.	**Tambahkan ->names('foods') pada Route::resource('/admin/foods', ...) di routes/web.php.**

Pesan Error / Gejala	Penyebab Utama	Solusi Instan 10 Detik
405 Method Not Allowed Http Exception	Method form HTML tidak sesuai dengan rute web.php (misal: Update tanpa spoofing PUT).	**Tambahkan @method('PUT') pada form edit, atau @method('DELETE') pada form hapus.**
SQLSTATE[23000]: Foreign key constraint fails saat migrate	Nama tabel di migration Food adalah 'food' (tunggal), atau urutan tabel order_details dijalankan sebelum foods.	**Pastikan Schema::create('foods', ...) berakhiran 's', dan buat urutan foods -> orders -> order_details.**
SQLSTATE[HY000]: General error: 1364 Field doesn't have default value	Kolom model diblokir mass assignment karena belum diizinkan.	**Tambahkan protected $guarded = ['id']; di dalam class Model bersangkutan.**
Foto 404 / Gambar Rusak Tidak Tampil di Browser	Belum membuat symlink shortcut antara storage internal dan folder public.	**Jalankan di terminal: php artisan storage:link**
Foto Tidak Tersimpan / Bernilai NULL di Database	Tag <form> lupa diberi atribut encoding berkas.	**Tambahkan atribut enctype="multipart/form-data" pada tag <form> create & edit.**
Class 'DB' not found atau Class 'Storage' not found	Lupa meng-import Facade Laravel di bagian atas controller.	**Tambahkan use Illuminate\Support\Facades\DB; atau use Illuminate\Support\Facades\Storage; di atas controller.**
Attempt to read property 'name' on null di Dashboard	Ada detail pesanan yang food_id-nya sudah terhapus di tabel foods.	**Gunakan null-safe operator di Blade: {{ $detail->food->name ?? 'Menu Dihapus' }}.**
Call to a member function store() on null	Memanggil $request->file('image')->store() padahal user tidak mengupload foto.	**Bungkus dengan pengecekan: if ($request->hasFile('image')) { ... }**
Vite manifest not found at: public/build/manifest.json	Asset Breeze belum di-build setelah instalasi.	**Jalankan di terminal: npm install && npm run build**
SQLSTATE[42S02]: Base table or view not found: 1146 Table 'pesanmakan.foods' doesn't exist	Database belum dimigrasi atau nama database di .env berbeda.	**Cek .env DB_DATABASE=pesanmakan lalu jalankan: php artisan migrate:fresh --seed**
Undefined variable $foods atau $orders di view Blade	Lupa mengirim variabel via compact() atau typo penulisan nama variabel di controller.	**Periksa controller: return view('...', compact('foods')); (pastikan tanpa tanda dollar di compact).**
Target class [FoodController] does not exist di routes/web.php	Lupa meng-import namespace controller di bagian atas routes/web.php.	**Tambahkan use App\Http\Controllers\FoodController; di baris atas routes/web.php.**
Maximum execution time of 30 seconds exceeded saat composer require	Koneksi internet lambat saat mengunduh Breeze.	**Jalankan di terminal: composer require laravel/breeze --dev --timeout=300**